

Gray-Scott - a walkthrough

Gray-Scott

Two chemicals, four lines of arithmetic, and a 768×768 grid where every single cell is computed by the same Mojo kernel on the Apple GPU.

U feeds itself back toward 1. V is a catalyst that eats U wherever both are present, and is itself removed at a fixed rate. That is the entire model — and depending on where two numbers sit, that same four-line kernel produces spots that divide like cells, worms that wriggle forever, spinning spirals, or a coral maze that never stops growing.

These documents

Chapter	What it covers
1. Where it comes from	Turing, Gray and Scott, and Pearson's map of the plane
2. The model	Two chemicals and four lines, in the code's own terms
3. Why this belongs on a GPU	The textbook case, checked property by property
4. The three kernels	Every kernel, line by line
5. A frame, end to end	The ping-pong, the substeps, and the loop
6. What to understand	What will surprise you, and what will bite

At a glance

Source	examples/grayscale/main.mojo, 693 lines
Grid	768 × 768 — 589,824 cells, one thread each
Per frame	20 substeps + 1 colour pass = 21 dispatches
What decides the pattern	two floats: feed and kill

The shortest possible summary

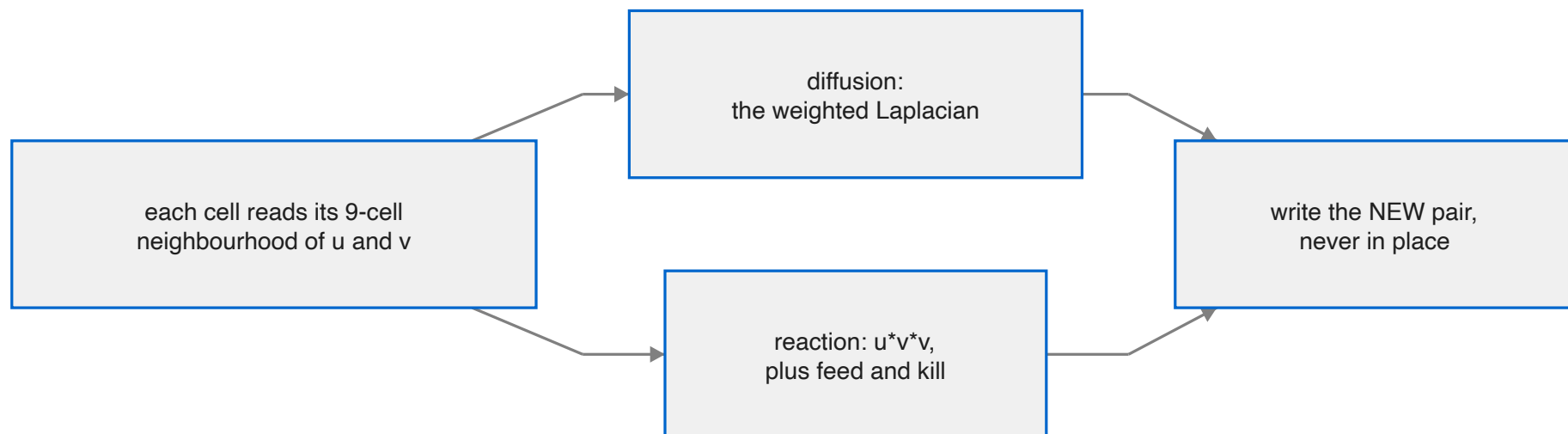
Every cell holds two numbers, `u` and `v`. Each step, each cell looks at its eight neighbours, and applies this:

$$\begin{aligned} du &= Du \cdot \nabla^2 u - u \cdot v^2 + \text{feed} \cdot (1 - u) \\ dv &= Dv \cdot \nabla^2 v + u \cdot v^2 - (\text{feed} + \text{kill}) \cdot v \end{aligned}$$

The first term in each is **diffusion** — spreading out. The $u \cdot v^2$ is **reaction** — V consumes U and makes more of itself, which is why a single speck of catalyst grows outward along its own edge rather than eating everything at once. The last term is **replenishment** for U and **removal** for V.

Diffusion alone smooths everything to a uniform grey. Reaction alone runs to completion and stops. Together, at the right rates, they never settle — and the shapes that come out are the ones biology makes.

Nothing in the kernel knows what a spot or a stripe is. The six presets in the file are six *points* on one continuous map, and moving `feed` by three thousandths puts the identical kernel in a different regime.



1. Where it comes from

Three ideas, forty years apart, and the code carries all three.

1952: Turing asks how a blob becomes an animal

Alan Turing's last published paper was not about computing. "**The Chemical Basis of Morphogenesis**" (1952) asked a question that had no good answer: how does a spherical, uniform embryo become a thing with a head, a tail and stripes? Where does the asymmetry come from, if the starting state is symmetric?

His answer was startling, and it is the reason this example exists.

Take two chemicals that react with each other and also diffuse. Diffusion is the great smoother — left alone it removes every difference and produces a uniform field. So the intuition is that diffusion plus reaction should smooth even faster.

Turing showed the intuition is wrong. If the two chemicals **diffuse at different rates**, a uniform state can become *unstable*: a tiny random fluctuation grows instead of dying, and the system settles into a stationary pattern with a characteristic wavelength. Spots. Stripes.

He called the chemicals **morphogens**, and the mechanism is now called a **Turing instability** — diffusion, the thing that destroys structure, being the thing that creates it.

The rate difference is not incidental, and it is in the code as two constants:

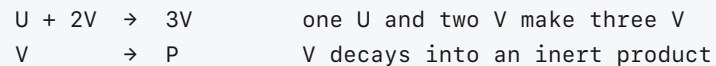
```
comptime DU = Float32(1.0)
comptime DV = Float32(0.5)
```

V diffuses at half U's rate. Set them equal and the patterns do not form.

1983: Gray and Scott build a reactor

Turing's paper was theory. Peter Gray and Stephen Scott, working on combustion and chemical engineering in the early 1980s, studied a concrete system: an **autocatalytic** reaction in a continuously stirred tank reactor — fresh reagent flowing in, product flowing out.

Autocatalytic means the product catalyses its own production. Their scheme is two steps:



The first line is the whole personality of the model, and it is one term in the kernel:

```
var uvv = u * v * v
```

$u * v * v$ — the rate depends on U being present *and* on V being present, squared. So **V can only grow where V already is**. A speck of catalyst in a sea of U does not consume the sea; it eats along its own boundary and spreads outward as a ring.

The reactor gives the other two terms. Feedstock arrives:

```
+ feed * (Float32(1.0) - u)
```

U is topped back up toward 1 everywhere, at rate `feed`. And everything drains out:

```
- (feed + kill) * v
```

V is removed at `feed + kill` — the `feed` part because the outflow carries it away, the `kill` part as its own decay.

That is the whole of the source comment, in the language of a reactor:

```
U feeds itself back toward 1, V is a catalyst that eats U wherever both are present and is itself removed at a fixed rate.
```

1993: Pearson maps the plane

The step that made this famous in graphics rather than chemistry was John Pearson's "**Complex Patterns in a Simple System**" (*Science*, 1993).

Pearson took the Gray-Scott equations, fixed everything except `feed` and `kill`, and swept those two across a plane — cataloguing what came out. The answer was a zoo. Small regions of the plane produce spots that divide like cells; adjacent regions produce stripes, or travelling waves, or spirals, or patterns that never stop rearranging.

The finding was not that the model makes patterns. It is that **one equation makes all of them**, and which one you get is a point on a two-dimensional map with sharp boundaries.

The example is built on exactly that:

```
# Six named regions of the Gray-Scott (feed, kill) plane -- the widely
# reproduced values this model is usually shown with, not a guess: get
# these even a few thousandths wrong and the field relaxes to a uniform
<!-- doccrate:keep-together:start -->

# grey instead of staying alive.
```

preset	feed	kill
gliders	0.014	0.054
bubbles	0.098	0.057
maze	0.029	0.057
worms	0.058	0.065
spirals	0.018	0.051
spots	0.030	0.062

Look at **maze** and **bubbles**: identical `kill`, and `feed` differing by 0.069. Or **maze** and **spots**: `feed` a thousandth apart, `kill` five thousandths apart, and one grows space-filling channels while the other makes dividing dots.

That sensitivity is why the values are looked up rather than invented, and the

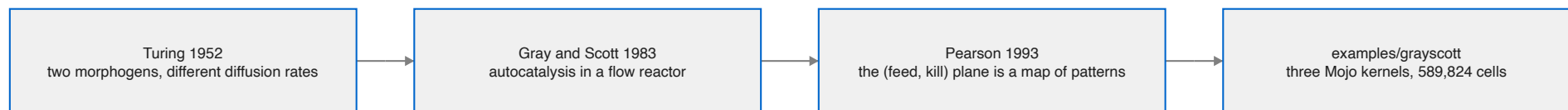
comment says what happens if you guess: *the field relaxes to a uniform grey instead of staying alive*. There is no error and no crash. The simulation simply stops being interesting, which is the most annoying kind of wrong.

What the code inherits from each

| from | what is in the file |

|:---|:---| | Turing, 1952 | two species diffusing at **different** rates — $DU = 1.0$, $DV = 0.5$ || Gray & Scott, 1983 | the $u*v*v$ autocatalysis, $feed \cdot (1-u)$ inflow, $(feed+kill) \cdot v$ outflow | | Pearson, 1993 | six presets as six points on one map, and live switching between them |

And one thing that is this fork's own: every cell of it runs as a Mojo kernel on the GPU, which is [chapter 3](#).



2. The model

You do not need chemistry to read this program, but you do need three ideas. Each is one term in the kernel, which is why the kernel is four lines.

What is being simulated

Two scalar fields on a 768×768 grid:

- **U** — the substrate. Starts at 1.0 everywhere: a full tank.
- **V** — the catalyst. Starts at 0.0 everywhere except a few seeded discs.

Both are held in $[0, 1]$. There is no velocity field, no pressure, no constraint linking distant cells. Two numbers per cell, and that is the entire state.

Idea one: diffusion, and the Laplacian that measures it

Diffusion spreads a substance from where there is more of it to where there is less. The rate at which a point's concentration changes is proportional to how much that point differs from its surroundings — which is what the **Laplacian** measures.

```
def _laplacian(f: Pointer[Float32, MutAnyOrigin], x: Int, y: Int) -> Float32:
    """The standard nine-point weighted Laplacian: corners a twentieth,
    edges a fifth, the centre minus one whole. The weights sum to zero,
    which is what makes this a rate of CHANGE rather than a blur -- a
    perfectly flat field produces zero here no matter what value it is
    flat at."""
```

```
0.05  0.2  0.05
0.2   -1.0 0.2
0.05  0.2  0.05
```

The docstring names the property that matters: **the weights sum to zero**. $0.05 \times 4 + 0.2 \times 4 - 1.0 = 0$.

That is the difference between a Laplacian and a blur. A blur's weights sum to one, so it returns an average — a flat field blurs to the same flat field, but the *value* comes back. These weights return **zero** for any flat field, whatever it is flat at. What comes back is not a concentration; it is a rate of change.

Get the weights wrong by a little and they no longer sum to zero, which means a flat field spontaneously gains or loses substance everywhere — and the simulation drifts in a way that looks like a physics bug rather than an arithmetic one.

The nine-point form — including diagonals at a quarter of the edge weight — is the standard discrete Laplacian for this model. A five-point version (edges only) works but is visibly more square: patterns grow along the axes because the operator itself prefers them.

Idea two: reaction, and why $u \cdot v^2$ is the whole personality

```
var uvv = u * v * v
```

One multiply chain, and it is the reason anything interesting happens.

The reaction is $U + 2V \rightarrow 3V$: V consumes U and turns it into more V. The rate depends on U being present, and on V being present **squared** — because two molecules of V have to meet one of U.

Three consequences, all visible on screen:

- **V cannot appear where V is absent.** $v = 0$ makes the term zero however much U is there. Seed a blank grid with U only and nothing ever happens.
- **Growth is along the boundary.** In the interior of a V patch, U has already been eaten, so $u \approx 0$ and the term is small. At the edge, both are present. So a disc of catalyst spreads as a *ring*.
- **It is self-limiting.** Consuming U removes the thing the reaction needs.

The source puts the same point in one sentence:

V can only eat U where both are already present, which is why a single catalyst speck grows outward along its own edge rather than consuming everything at once.

That is why the seed kernel does what it does:

```
u[unsafe_offset=idx] = Float32(0.5)
v[unsafe_offset=idx] = Float32(1.0)
```

U half-consumed, V present -- exactly what a real drop of the second chemical looks like the instant after it lands.

Not `u = 0`. Leaving some U inside the disc gives the reaction something to work with immediately, which is what a real drop looks like a moment after landing.

Idea three: feed and kill, the two numbers that decide everything

```
var du = DU * _laplacian(u_in, x, y) - uvv + feed * (Float32(1.0) - u)
var dv = DV * _laplacian(v_in, x, y) + uvv - (feed + kill) * v
```

Read the two lines as a pair. `uvv` appears **negative in du and positive in dv** — the same quantity, moved from one species to the other. Whatever U loses, V gains, which is what a chemical reaction is.

Then:

- **feed * (1 - u)** replenishes U everywhere, and note the `(1 - u)`: the further U is below 1, the harder it is topped up. Where U is untouched the term is zero. It is a pull toward full, not a constant inflow.
- **(feed + kill) * v** removes V everywhere, in proportion to how much is there. The `feed +` part is the outflow of a reactor carrying V away with everything else; `kill` is V's own decay on top.

Now the balance. V is created only where the pattern already is, and destroyed *everywhere*. So a V region survives only if its edge produces V faster than its interior loses it — and that comparison is exactly what `feed` and `kill` set.

- **Kill too high** relative to feed: removal wins, V dies out, the field relaxes to `U = 1` everywhere.
- **Kill too low**: V's growth outruns removal, V floods the grid, and the field becomes uniform the other way.
- **Between them** is a narrow band where neither wins, and the boundary between the two states becomes a permanent, moving structure.

Every pattern this program draws lives in that band, which is why the presets sit within a few thousandths of one another and why the file bothers to say that guessing produces *"a uniform grey instead of staying alive."*

The clamps

```
if nu < Float32(0.0):  
    nu = Float32(0.0)  
elif nu > Float32(1.0):  
    nu = Float32(1.0)
```

Concentrations are physically in $[0, 1]$ and the explicit Euler step ($u + du \cdot dt$) can overshoot. The clamp is not a numerical fudge so much as a statement about what these numbers are: a negative concentration is not a small error, it is a meaningless value that would feed straight back into $u \cdot v \cdot v$ next step and never recover.

$DT = 1.0$ looks bold, but the units are the model's own — the diffusion constants and rates are all scaled to it, and this is the standard discrete form.

Only V is drawn

```
"""V alone carries the picture: everywhere U has not been eaten sits  
at the same becalmed 1.0, so V's shape IS the pattern."""
```

Both fields are simulated; only one is displayed, and the reason is that U is almost entirely uninformative. Away from the pattern it is pinned at 1.0 by the feed term, so a picture of U is a flat field with holes in it. V is zero except where something is happening, so V's shape is exactly the structure.

3. Why this belongs on a GPU

The other examples in this collection spend their arguments explaining why a GPU is the *wrong* tool — alpha-beta's cutoffs diverge, the chaos game is a recurrence, the logistic map cannot be vectorised. Gray-Scott is the opposite case, and it is worth being precise about why, because "it's a grid, so it's parallel" is not the reason.

The six properties, checked one at a time

1. One output per input, and no output depends on another

Within a single step, cell (x, y) 's new value depends on the **old** values of its neighbourhood and on nothing else. No cell needs to know what any other cell computed *this* step.

That is a pure map, and it is the strongest form of parallelism there is: 589,824 independent computations, in any order, with any scheduling.

The code enforces it structurally rather than by care:

```
def graycott_kernel(u_out, v_out, u_in, v_in, feed, kill):
    """One reaction-diffusion step, one cell, reading the OLD pair and
    writing the NEW one -- never in place, since every neighbour read
    needs last step's values, not whatever this thread has already
    written this step."""
```

Separate in and out buffers. Write in place and a thread reads neighbours that another thread has already updated — some old, some new, depending entirely on GPU scheduling. The result still looks like a reaction-diffusion system and is different on every run. Fluid's Jacobi kernel documents the identical hazard, and both defend against it the same way: two buffers, swapped.

2. The control flow is identical in every thread

```
var idx = Int(global_idx.x)
if idx < CELLS:
    ... nine reads, twenty-odd flops, two clamped writes ...
```

One bounds check, then straight-line arithmetic. **No branch depends on the data.** The `_clampi` calls are min/max, which compile to selects rather than jumps; the output clamps likewise.

This is the property that separates a good GPU workload from a merely parallel one. Threads in a warp execute together; when they take different paths the hardware runs both and masks. Here there are no different paths — every one of 589,824 threads runs the same instruction sequence with different numbers, which is precisely what the hardware is built for.

Compare Othello's alpha-beta, where every thread would hit its cutoff at a different moment.

3. The stencil is fixed, small, and local

Nine reads at a compile-time-known offset pattern. That means:

- **The addresses are computable, not looked up.** No indirection, no pointer chasing, no index array.
- **Neighbouring threads read overlapping data.** Thread n and thread $n+1$ share six of their nine cells. On any GPU with a cache, most of those reads are already resident — the effective memory traffic is far below nine loads per cell.
- **The working set is a few rows.** A block of 256 consecutive cells touches three rows of 256-ish values; that is kilobytes, not megabytes.

4. There is no reduction and no global communication

This is the point that distinguishes Gray-Scott from Fluid, and it is the most interesting comparison in the collection.

Fluid has an **incompressibility constraint**, which is global: the velocity field must be divergence-free *everywhere at once*, and you cannot satisfy a global constraint with purely local updates. That is why Fluid needs a pressure solve, and why the solve is thirty Jacobi iterations — thirty dispatches propagating information across the grid, each dependent on the last.

Gray-Scott has **no global constraint at all**. Nothing has to be conserved across the grid, nothing has to be solved. Information travels one cell per step, and that is not a limitation to be worked around — it is the physics.

	Fluid	Gray-Scott
per frame	~35 dispatches	21 dispatches
of which a solver	30 Jacobi sweeps	none
what couples cells	a global constraint	one cell of diffusion
buffers	twelve	five
synchronisation inside a step	required	none

Both are 60 fps. Fluid's frame is dominated by *launch overhead* across many small dependent dispatches; Gray-Scott's twenty substeps are each real work over 589,824 cells.

5. The arithmetic-to-memory ratio is favourable

Per cell, per step: 18 float reads (nine each for U and V), about 24 floating-point operations, and 2 writes. Roughly one flop per byte moved — which for a stencil with this much neighbour reuse means the caches do most of the work and the arithmetic units stay busy.

A stencil that read *one* value per cell and did one multiply would be purely memory-bound and would gain far less from a GPU.

6. The batch is large and the result is small

589,824 threads per dispatch, and the only thing that comes back to the host is one finished image, once per frame. Twenty substeps happen entirely on the device with nothing crossing the bus between them.

The one dependency, and where it goes

Steps are strictly ordered: step $n+1$ needs all of step n . That dependency is real and unavoidable.

It costs nothing here because it maps onto a boundary the hardware already has. A dispatch completes before the next begins, so the step boundary *is* the dispatch boundary — no barrier, no fence, no code.

```
for _step in range(SUBSTEPS // 2):  
    ctx.enqueue_function(gs_kern, u2, v2, u, v, feed, kill, ...)  
    ctx.enqueue_function(gs_kern, u, v, u2, v2, feed, kill, ...)
```

Twenty enqueues, no synchronise between them. The CPU does not wait; it queues the batch and blocks once, later, before reading the frame.

What it is *not* good at, honestly

Two things are worth saying so the argument does not overreach.

It is redundant at the edges of interest. Large regions of the grid are becalmed — U pinned at 1, V at 0 — and those cells are computed at full cost every step. An adaptive method would skip them. It would also branch per cell, which is exactly what a GPU punishes, so the uniform version wins anyway. That is a real trade: the GPU version does *more* arithmetic than necessary and is still far faster.

It is not latency-friendly. Twenty substeps per frame exist because one step per frame would make the pattern evolve too slowly to watch. That is a throughput problem, which suits a GPU — but if you needed a single step *now*, the dispatch overhead would dominate, exactly as it does for Othello's 81 μ s alpha-beta.

The summary

what the hardware wants	what Gray-Scott does
independent work items	589,824 cells, no intra-step coupling
uniform control flow	one bounds check, then straight-line arithmetic
local, predictable memory	a fixed 9-cell stencil with 6-cell overlap between neighbours
no reduction or atomics	none — there is no global constraint to satisfy
a large batch, a small result	one image per frame; 20 substeps never leave the device
arithmetic to hide latency	~24 flops per cell over cached reads

Six for six, which is unusual. This is the workload the hardware was designed around, and the useful thing about reading it beside Othello and the fern examples is seeing what the exceptions actually look like.

4. The three kernels

Three kernels, all ordinary Mojo `def`s compiled through this fork's AIR backend. There is no shading language anywhere.

They share one shape:

```
var idx = Int(global_idx.x)
if idx < CELLS:
  ...
```

One thread per cell, flat indexing, a bounds guard because 589,824 cells round up to 2,304 blocks of 256.

The helpers

`_at` — a clamped read

```
def _at(f: Pointer[Float32, MutAnyOrigin], x: Int, y: Int) -> Float32:
    return f[unsafe_offset=_clampi(y, 0, HEIGHT - 1) * WIDTH
            + _clampi(x, 0, WIDTH - 1)]
```

This is Fluid's `_at`, copied verbatim, and the commit says why that is the right kind of reuse:

edge cells are never special-cased, only the neighbour reads they make are clamped — fluid's own `_at`, copied verbatim.

The distinction in that sentence is the whole discipline. There is **no branch for edge cells** — every one of the 589,824 threads runs the identical code. It is only the *read* that clamps, and a clamp is two min/max operations, which compile to selects rather than jumps.

The alternative — `if (x > 0 && y > 0 && ...)` around the stencil — would put a data-dependent branch in every thread, and the ~3,000 edge cells would diverge from the 587,000 interior ones. For 0.5% of the grid, you would pay divergence on all of it.

Physically the clamp is a reflecting boundary: a cell at the edge sees itself where its missing neighbour would be, so nothing leaks out and nothing wraps around.

`_laplacian` — nine reads, weights summing to zero

```
s += _at(f, x - 1, y - 1) * Float32(0.05)
s += _at(f, x,      y - 1) * Float32(0.2)
...
s += _at(f, x,      y)      * Float32(-1.0)
```

Written out flat rather than as a loop over offsets. Nine `_at` calls, all `@always_inline`, all with compile-time-known offsets — so the compiler folds the index arithmetic and the whole thing becomes nine loads and nine multiply-adds with no loop overhead and no branch.

A loop over an offset table would be shorter to read and would introduce an indirection through that table, per cell, per step, 589,824 times over.

[Chapter 2](#) covers why the weights sum to zero.

1. `grayscale_kernel` — twenty of the twenty-one dispatches

```
var u = _at(u_in, x, y)
var v = _at(v_in, x, y)
var uvv = u * v * v
var du = DU * _laplacian(u_in, x, y) - uvv + feed * (Float32(1.0) - u)
var dv = DV * _laplacian(v_in, x, y) + uvv - (feed + kill) * v
var nu = u + du * DT
var nv = v + dv * DT
```

Four lines of model and two of Euler integration. Everything in [chapter 2](#) is here and nowhere else.

Two things about its *shape* rather than its physics:

feed and kill are kernel arguments, not constants. That is what lets the number keys switch preset live without recompiling — the same compiled kernel, dispatched with different scalars. Baking them in as `comptime` would have been marginally faster and would have made six kernels out of one.

In and out are separate parameters, which is what makes the ping-pong in [chapter 5](#) a matter of argument order rather than of copying.

2. seed_kernel — a disc of catalyst

```
var dx = x - cx
var dy = y - cy
if dx * dx + dy * dy <= r2:
    u[unsafe_offset=idx] = Float32(0.5)
    v[unsafe_offset=idx] = Float32(1.0)
```

Every cell tests whether it is inside the disc; most are not and write nothing. That is 589,824 threads to paint a disc of radius 14 — about 600 cells. Wildly redundant, and correct: a dispatch is one launch whatever it touches, and the alternative is a separate code path for a job that happens a handful of times per session.

It is used for **both** the initial seeding and the mouse, which is why dragging feeds the pattern by hand with exactly the physics the simulation started with.

The docstring records a dialect constraint worth knowing:

Int32, not Int: a kernel argument must be fixed-width to conform to DevicePassable -- plain Int (and UInt) do not, a constraint that only bites here because this is the one kernel in the file passing a bare integer rather than a Pointer or a Float32.

`Int` is pointer-width and its size is a host-side property; a kernel argument has to have a size both sides agree on. The other two kernels never hit it because their arguments are pointers and `Float32`s.

3. color_kernel — the only one that is not chemistry

```
var tt = t * Float32(2.2) + phase
```

```
var r = Float32(0.5) + Float32(0.5) * cos(TAU * (tt + Float32(0.10)))
var g = Float32(0.5) + Float32(0.5) * cos(TAU * (tt + Float32(0.45)))
var b = Float32(0.5) + Float32(0.5) * cos(TAU * (tt + Float32(0.65)))
var shade = t * Float32(3.0)
```

A **cosine palette**: three cosines a third of a cycle apart, each mapped from $[-1, 1]$ into $[0, 1]$. Every value of `t` gets a colour, the ramp is smooth and cyclic, and there is no lookup table — three cosines and three adds.

Mandelbrot's own trick here tuned to a different register — deep water climbing through coral toward bone.

Two details beyond the palette:

phase drifts. It advances 0.0015 per frame, so the colour mapping rotates slowly under a pattern that is itself changing on a different timescale. The picture never looks static even when the chemistry has nearly settled.

shade is separate, and it is doing real work:

a separate shade term darkens the untouched sea toward black rather than letting the cosines paint a colour where nothing is actually happening.

At `t = 0` the three cosines still evaluate to *something* — a cosine palette has no black. Without `shade`, the empty regions of the grid would come out a uniform mid-tone, and the pattern would read as texture on a coloured field rather than as structure in the dark. `t * 3.0`, clamped at 1, means anything below $V = 0.33$ is progressively darkened toward black.

The packing is direct:

```
# BGRA8Unorm little-endian: byte0=B, byte1=G, byte2=R, byte3=A.
return b | (g << 8) | (r << 16) | (UInt32(255) << 24)
```

The kernel writes the exact 32-bit words the drawable's texture wants. No shader, no sampler, no conversion pass — `replaceRegion:` takes this buffer as it stands.

The count

kernel	dispatches per frame
grayscale	20 — ten ping-pong pairs
color	1
seed	0, except when seeding or dragging
	21 fixed

5. A frame, end to end

One file, one loop. This chapter walks a frame from the mouse to the drawable.

Before the loop: five buffers, three kernels

```
var u = ctx.enqueue_create_buffer[DType.float32](CELLS)
var v = ctx.enqueue_create_buffer[DType.float32](CELLS)
var u2 = ctx.enqueue_create_buffer[DType.float32](CELLS)
var v2 = ctx.enqueue_create_buffer[DType.float32](CELLS)
var dev = ctx.enqueue_create_buffer[DType.uint32](CELLS)

var gs_kern = ctx.compile_function[grayscale_kernel]()
var seed_kern = ctx.compile_function[seed_kernel]()
var color_kern = ctx.compile_function[color_kernel]()
```

Five device buffers — two pairs plus the frame — allocated once, and three kernels compiled once. Nothing is allocated per frame.

Compare Fluid's twelve buffers and six kernels: the difference is entirely the pressure solve that Gray-Scott does not need.

Then the initial state:

```
ctx.enqueue_memset(u, Float32(1.0))
ctx.enqueue_memset(v, Float32(0.0))
for _i in range(SEED_BLOBS):
    ...
    ctx.enqueue_function(seed_kern, u, v, rx0, ry0, SEED_R2, ...)
```

A full tank of U, no V, then six random discs of catalyst.

There is a comment where a helper function would be:

```
# Not a nested function: a DeviceContext and its buffers captured by a  
# closure is untested ground, and this is called from exactly two  
# places, so inlining both times is the safer six lines.
```

Six duplicated lines against an untested capture, in a dialect where the compiler's closure handling has surprised people before. The comment records that the choice was made rather than missed, which is what stops someone "tidying" it.

Step 1: events, into flags

```
class GrayScottView(NSView):
```

Handlers set bits and nothing else:

```
comptime CMD_CLICK = 1  
comptime CMD_PAUSE = 2  
comptime CMD_RESET = 4  
comptime CMD_QUIT = 8  
comptime CMD_PRESET = 16  
comptime CMD_SAVE = 32
```

Bits rather than booleans, so two commands arriving in one frame cannot lose each other. It is the house rule from Fluid and Othello, and the commit states the reason:

input arrives through a class whose handlers only set flags, so every Metal and GPU call still happens on the thread that owns them.

A `DeviceContext` lives in a local in `main`, and a Cocoa callback cannot reach a local. So the pump owns the GPU and the handlers own nothing.

Step 2: acting on them

```
var pending = g_cmd()[]
if pending != 0:
    g_cmd()[] = 0
```

Note `var`, and note the order: read the flags **out** of the global, then clear. `let` binds to a place in this dialect, so `let pending = g_cmd()[]` would be a live view — clearing the global would empty it and every command would evaporate. That exact bug cost Othello three debugging sessions and is written up in `CLAUDE.md`.

The interesting branch is the preset switch:

```
if (pending & CMD_PRESET) != 0:
    preset = g_want_preset()[]
    feed = preset_feed(preset)
    kill = preset_kill(preset)
    win.title = "Gray-Scott - " + preset_name(preset)
```

Two floats change, and the next dispatch is in a different regime. **The kernel is not recompiled and the field is not reset** — the existing pattern carries on under new rules, which is far more instructive than a fresh start: you watch a maze reorganise itself into worms.

Step 3: twenty substeps, ping-ponged

```
if not paused:
    for _step in range(SUBSTEPS // 2):
        ctx.enqueue_function(gs_kern, u2, v2, u, v, feed, kill, ...)
        ctx.enqueue_function(gs_kern, u, v, u2, v2, feed, kill, ...)
```

Read the argument order. The first call has `u2, v2` as destination and `u, v` as source; the second reverses them. Ten pairs, twenty steps, and the settled state ends up back in `u, v`.

That last part is the point of the constant:

```
comptime SUBSTEPS = 20
"""Simulation steps per drawn frame. Kept EVEN on purpose: the ping-pong
below always leaves the settled state back in (u, v), so the colour
kernel never has to ask which buffer is current."""
```

Make it odd and the answer finishes in `u2`, `v2`, the colour kernel reads `v`, and you get the state from one step ago — every frame, consistently, which looks like nothing at all is wrong.

The alternative designs are worth naming. You could track a `current` flag and branch on it; you could copy back after each step. The even-pair loop costs neither: no branch, no copy, and the invariant is maintained by the loop's structure.

Twenty steps rather than one because a single step at this grid size is a barely visible change. Twenty makes the pattern evolve at a watchable rate, and the whole batch is enqueued without a single synchronise.

Step 4: colour, and the one blocking point

```
ctx.enqueue_function(color_kern, dev, v, phase, grid_dim=(GRID), block_dim=(BLOCK))
ctx.synchronize()
with dev.map_to_host() as pix:
    var src = pix.unsafe_ptr()
    for k in range(CELLS):
        bgra[unsafe_offset=k] = src[unsafe_offset=k]
```

`ctx.synchronize()` is the **only** blocking point in the frame. Twenty-one dispatches were queued; this is where the CPU waits for all of them at once.

Note the colour kernel reads `v` — never `v2` — which is what the even substep count guarantees.

Step 5: the snapshot, if asked

```
# Save here, not later: `bgra` is exactly what is about to be  
# presented, so the file and the window cannot disagree.
```

Fluid's comment and Fluid's `save_png`, carried over — and the file says it is a copy rather than an import:

example folders are self-contained by this tree's own convention, so this is a copy, not an import.

The headless door does something better than exiting cleanly:

```
# A headless run (GRAYSCOTT_FRAMES) leaves one PNG of its last  
# frame behind -- a CI run produces something a person can  
# actually look at, not just an exit code.
```

The commit is explicit that this is what verification meant here:

Verified by actually looking at the output, not just a clean exit: a saved PNG shows a textbook maze pattern after 12,000 steps, and switching to the worms preset produces visibly different physics (rings forming from each seed rather than maze's space-filling channels) — proof the parameter wiring changes what is actually computed, not just a label.

That is the right test for this program. A clean exit proves the dispatches ran. It does not prove `feed` and `kill` reached the kernel — a preset switch that only changed the window title would pass every automated check.

Step 6: present

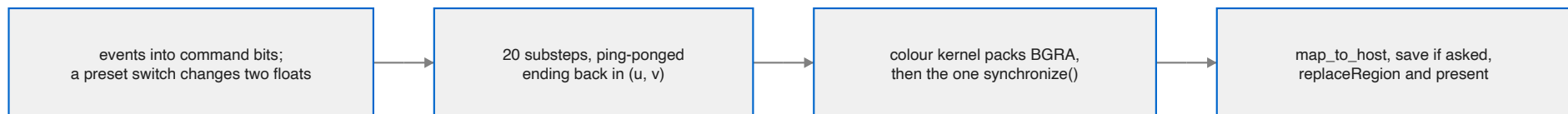
```
var drawable = Obj["CAMetalLayer"](layer.addr()).nextDrawable()  
if drawable.id != 0:  
    var tex = Obj["CAMetalDrawable"](drawable.addr()).texture()  
    _ = send[ObjCObject, "replaceRegion:mipmapLevel:withBytes:bytesPerRow:"](  
        tex, region, Int(0), bgra.unsafe_bitcast[NoneType](), Int(WIDTH * 4))  
    var cb = send[ObjCObject, "commandBuffer"](queue)
```

```
_ = send[ObjCObject, "presentDrawable:"](cb, drawable.ptr())
_ = send[ObjCObject, "commit"](cb)
```

The colour kernel already produced BGRA8, so the bytes go straight onto the drawable's texture. No shader, no sampler, no render pass — which is the first of the three house rules the commit names.

`send` rather than `msg_send` because the concrete classes behind `MTLTexture` and `MTLCommandQueue` are private, so there is no public class name to check against.

Note the `if drawable.id != 0` — `nextDrawable` returns nil when the pool is exhausted, and skipping the frame is correct.



6. What to understand

Eight things about this example are not obvious from watching it.

1. Two numbers decide which pattern you get

`feed` and `kill`, and nothing else. The same four-line kernel produces gliders, bubbles, maze, worms, spirals or spots depending on where those two sit — and the presets are a few thousandths apart.

get these even a few thousandths wrong and the field relaxes to a uniform grey instead of staying alive

Which is why the values are looked up rather than invented. The failure has no error and no crash; the simulation just stops being interesting.

Switching preset changes two floats and nothing else — not the kernel, not the field — so you watch one regime reorganise into another.

2. Diffusion is what makes the pattern, not what destroys it

The counter-intuitive core of Turing's 1952 result. Diffusion smooths, so diffusion plus reaction ought to smooth faster. It does not: when the two species diffuse at **different rates**, a uniform state becomes unstable and structure grows out of noise.

```
comptime DU = Float32(1.0)
comptime DV = Float32(0.5)
```

Set those equal and the patterns do not form.

3. $u * v * v$ is the whole personality

V can only grow where V already is, so a catalyst speck spreads as a *ring* along its own boundary rather than consuming the grid. In the interior of a patch U is already eaten, so the term is small; at the edge both are present.

Everything the program draws follows from that one product.

4. The Laplacian's weights sum to zero

```
0.05  0.2  0.05
0.2   -1.0 0.2
0.05  0.2  0.05
```

That is what makes it a rate of *change* rather than a blur. A flat field returns exactly zero, whatever it is flat at. Weights that sum to something else make a flat field gain or lose substance everywhere, and the drift looks like a physics bug rather than an arithmetic one.

5. Never write in place

```
"""reading the OLD pair and writing the NEW one -- never in place, since
every neighbour read needs last step's values, not whatever this thread
has already written this step."""
```

Write into the buffer you are reading and threads see a mixture of old and new neighbours, decided by GPU scheduling. It still looks like reaction-diffusion and differs on every run. Fluid's Jacobi kernel documents the same hazard and uses the same defence: two buffers, swapped.

6. SUBSTEPS must be even

Ten ping-pong pairs leave the settled state back in `u`, `v`, so the colour kernel never has to ask which buffer is current. Make it odd and the answer finishes in `u2`, `v2`, the colour kernel reads `v`, and you display the state from one step ago — every frame, consistently, which looks like nothing is wrong at all.

7. Clamp the reads, not the cells

```
def _at(f, x, y) -> Float32:  
    return f[unsafe_offset=_clampi(y, 0, HEIGHT - 1) * WIDTH  
            + _clampi(x, 0, WIDTH - 1)]
```

edge cells are never special-cased, only the neighbour reads they make are clamped

Every one of 589,824 threads runs identical code; only the address arithmetic clamps, and a clamp is a select rather than a branch. Guarding the *cells* instead would make 0.5% of the grid diverge from the rest, and you would pay for it across all of it.

8. The verification was looking at the picture

Verified by actually looking at the output, not just a clean exit... proof the parameter wiring changes what is actually computed, not just a label.

A clean exit proves the dispatches ran. It does not prove `feed` and `kill` reached the kernel — a preset switch that only updated the window title would pass every automated check. Two presets producing *visibly different physics* is what proves the wiring.

`GRAYSCOTT_FRAMES=N` leaves a PNG behind so a headless run produces something a person can look at rather than an exit code.

Things that will bite: the physics

If you change...	...this happens
DU and DV to be equal	no Turing instability; the patterns never form
the Laplacian weights so they do not sum to zero	a flat field drifts; looks like a physics bug
feed or kill by a few thousandths	a different pattern, or a uniform grey
the shade term out of the colour kernel	empty regions become a flat mid-tone, not black

Things that will bite: the machinery

If you change...	...this happens
SUBSTEPS to an odd number	the display shows the state from one step ago, always
the kernel to write in place	nondeterministic results, no error, no symptom
_at's clamp to a per-cell branch	the edge diverges from the interior, warp-wide
Int32 to Int on a kernel argument	it is not DevicePassable; the kernel will not build
a synchronize() between substeps	the batch splits and 20 dispatches are paid for separately

Controls

drag	paint catalyst under the cursor — the same seed kernel the simulation started with
1–6	gliders, bubbles, maze, worms, spirals, spots — live
r	reseed with fresh blobs at the current parameters
space	pause
q / esc	quit
GRAYSCOTT_FRAMES=N	render N frames unfocused, save a PNG, exit

And beside the others

example	what the GPU is for	dispatches per frame
mandelbrot	one independent value per pixel	1
grayscale	a local stencil, no global constraint	21
fluid	a <i>global</i> constraint needing a solver	~35
fernwind	many short independent random walks, with atomics	2
othello	independent playouts, one batch per move	1

Gray-Scott sits between mandelbrot and fluid, and reading the three together is the clearest way to see what the middle of that range costs.